

Manual Técnico

Realizado por: Manuel Camacho Font

05/03/2026

PROGRAMACIÓN

ÍNDICE

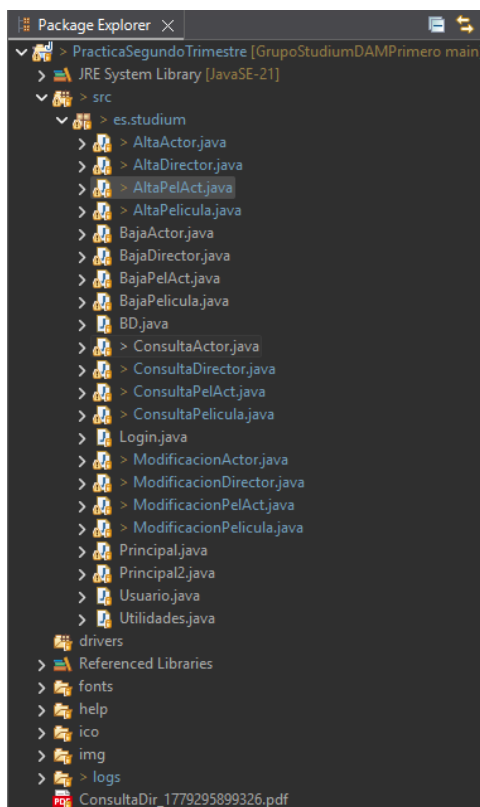
Presentación de la aplicación	Página 2
Herramientas Agile	Página 3
Diseño Base de Datos	Página 4
Construcción Base de Datos	Páginas 5 hasta 6
Usuario Base de Datos	Página 6
Diseño Funcional	Páginas 7 hasta 16
Repositorio GitHub	Página 17
Ejecución y Pruebas	Páginas 17 hasta 26
Referencias	Página 26

APLICACIÓN

Esta aplicación está diseñada en JAVA con la librería AWT, y se trata de un gestor de información conectado a una base de datos con el que podemos realizar las cuatro operaciones del CRUD (Create, Read, Update, Delete). Su nombre es Cinemateca por el hecho de que los datos con los que vamos a trabajar son del mundo del cine, trabajaremos con *Directores, Películas, Actores y la relación entre estos últimos*.

La Base de Datos que hemos conectado (usando el conector de tipo 4 JDBC) con la aplicación ha sido creada en MySQL server, actualmente existen dos tipos de usuarios con acceso a dicha Base de Datos, uno básico que solo tendrá permisos de INSERT en nuestras tablas y visualmente solamente dispondrá de esas opciones, y un usuario administrador con los permisos de INSERT, SELECT, UPDATE y DELETE.

Esta aplicación ha sido desarrollada en eclipse, con JDK21. Para la gestión de sus clases he creado una clase general llamada BD por motivos de organización y escalabilidad en la que guardo datos de conexión y sentencias SQL, una clase llamada Usuario en la que se controla y gestiona las funciones que puede realizar cada Usuario tanto para acceder al sistema como en el manejo de la aplicación y luego ya tenemos las vistas de Login, Principales y clases específicas para cada desempeño del CRUD. He añadido una clase llamada “Utilidades” con pequeñas funciones que he reutilizado en las diferentes vistas.



HERRAMIENTAS AGILE

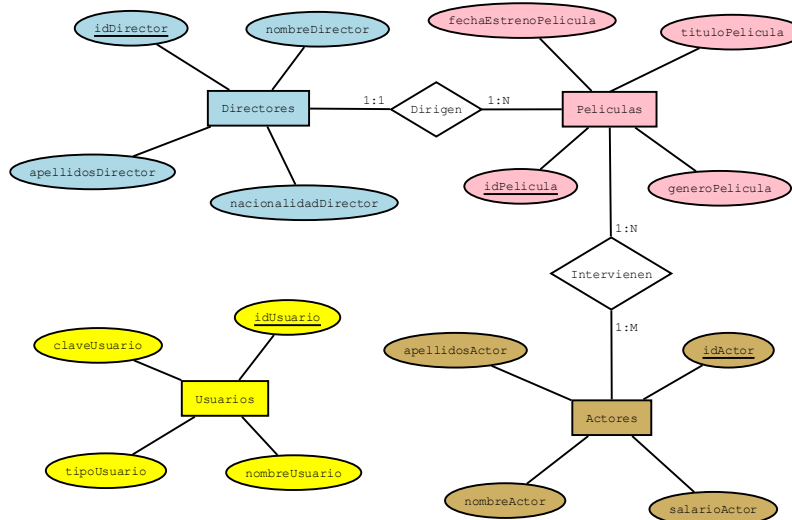
Para la organización y la gestión de este proyecto he optado por el uso de una herramienta que ofrece GitHub, en la que puedes anotar diferentes tareas dentro de tu proyecto. Planteé una estructura organizada a seguir y luego he ido anotando mejoras a realizar, bugs que arreglar e ideas que iban apareciendo.

☐	☒	Revisión y modificaciones #10 · by ManuelCamachoFont was closed on Mar 5
☐	☒	CRUD del resto de tablas #9 · by ManuelCamachoFont was closed on Mar 5
☐	☒	CRUD Director #8 · by ManuelCamachoFont was closed on Mar 5
☐	☒	Clases generales Usuario, BD y Login #7 · by ManuelCamachoFont was closed on Mar 5
☐	☒	Ventana principal y ventana principal2 #6 · by ManuelCamachoFont was closed on Mar 5
☐	☒	Usuario para la Base de Datos #5 · by ManuelCamachoFont was closed on Mar 5
☐	☒	Construcción de la Base de Datos #4 · by ManuelCamachoFont was closed on Mar 5
☐	☒	Diseño de la base de datos #3 · by ManuelCamachoFont was closed on Mar 5
☐	☒	Bocetos #2 · by ManuelCamachoFont was closed on Mar 5
☐	☒	Recopilar ideas #1 · by ManuelCamachoFont was closed on Mar 5

☐	☒	Validaciones #21 · by ManuelCamachoFont was closed now
☐	☒	Verificacion de duplicados #19 · by ManuelCamachoFont was closed 1 minute ago
☐	☒	Trim #18 · by ManuelCamachoFont was closed 1 minute ago
☐	☒	Float #17 · by ManuelCamachoFont was closed last week
☐	☒	Boton ConsPelAct #16 · by ManuelCamachoFont was closed last week
☐	☒	Imagenes #15 · by ManuelCamachoFont was closed 1 minute ago
☐	☒	Organizar búsquedas #13 · by ManuelCamachoFont was closed last week
☐	☒	Documentación y organización #12 · by ManuelCamachoFont was closed on Mar 5
☐	☒	Pruebas #11 · by ManuelCamachoFont was closed on Mar 5

DISEÑO DE BASE DE DATOS

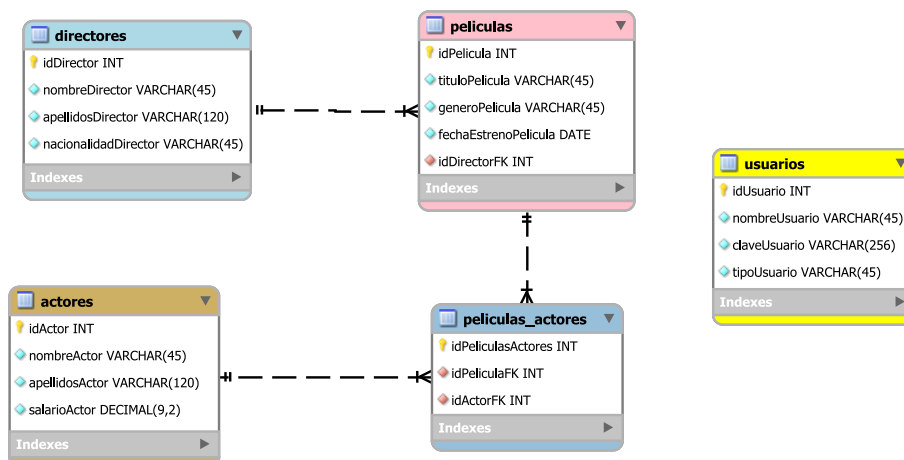
Diagrama Entidad – Relación



Esquema Relacional

```
directores(#idDirector, nombreDirector, apellidosDirector, nacionalidadDirector);
películas(#idPelícula, tituloPelícula, generoPelícula, fechaEstrenoPelícula, idDirectorFK);
actores(#idActor, nombreActor, apellidosActor, salarioActor);
películas_actores(#idPelículasActores, idPelículaFK, idActorFK);
usuarios(#idUsuario, nombreUsuario, claveUsuario, tipoUsuario);
```

Diagrama Workbench



CONSTRUCCIÓN DE BD

```
1      -- Crear BD
2 •    CREATE DATABASE p_cine
3      CHARSET utf8mb4
4      COLLATE utf8mb4_spanish2_ci;
5      -- Activar BD
6 •    USE p_cine;
7      -- Crear tabla directores
8 •    CREATE TABLE directores (
9          idDirector INT AUTO_INCREMENT,
10         nombreDirector VARCHAR(45) NOT NULL,
11         apellidosDirector VARCHAR(120) NOT NULL,
12         nacionalidadDirector VARCHAR(45) NOT NULL,
13         PRIMARY KEY (idDirector)
14     );
15     -- Crear tabla peliculas
16 •    CREATE TABLE peliculas (
17         idPelicula INT AUTO_INCREMENT,
18         tituloPelicula VARCHAR(45) NOT NULL,
19         generoPelicula VARCHAR(45) NOT NULL,
20         fechaEstrenoPelicula DATE NOT NULL,
21         idDirectorFK INT NOT NULL,
22         PRIMARY KEY (idPelicula),
23         FOREIGN KEY (idDirectorFK)
24             REFERENCES directores (idDirector)
25     );
26     -- Crear tabla actores
27 •    CREATE TABLE actores (
28         idActor INT AUTO_INCREMENT,
29         nombreActor VARCHAR(45) NOT NULL,
30         apellidosActor VARCHAR(120) NOT NULL,
31         salarioActor DECIMAL(9 , 2 ) NOT NULL,
32         PRIMARY KEY (idActor)
33     );
```

```
34      -- Crear tabla peliculas_actores
35 • ○ CREATE TABLE peliculas_actores (
36         idPeliculasActores INT AUTO_INCREMENT,
37         idPeliculaFK INT NOT NULL,
38         idActorFK INT NOT NULL,
39         PRIMARY KEY (idPeliculasActores),
40         FOREIGN KEY (idPeliculaFK)
41             REFERENCES peliculas (idPelicula),
42         FOREIGN KEY (idActorFK)
43             REFERENCES actores (idActor)
44     );
45      -- Crear tabla usuarios
46 • ○ CREATE TABLE usuarios (
47         idUsuario INT AUTO_INCREMENT,
48         nombreUsuario VARCHAR(45) NOT NULL,
49         claveUsuario VARCHAR(256) NOT NULL,
50         tipoUsuario VARCHAR(45) NOT NULL,
51         PRIMARY KEY (idUsuario)
52     );
```

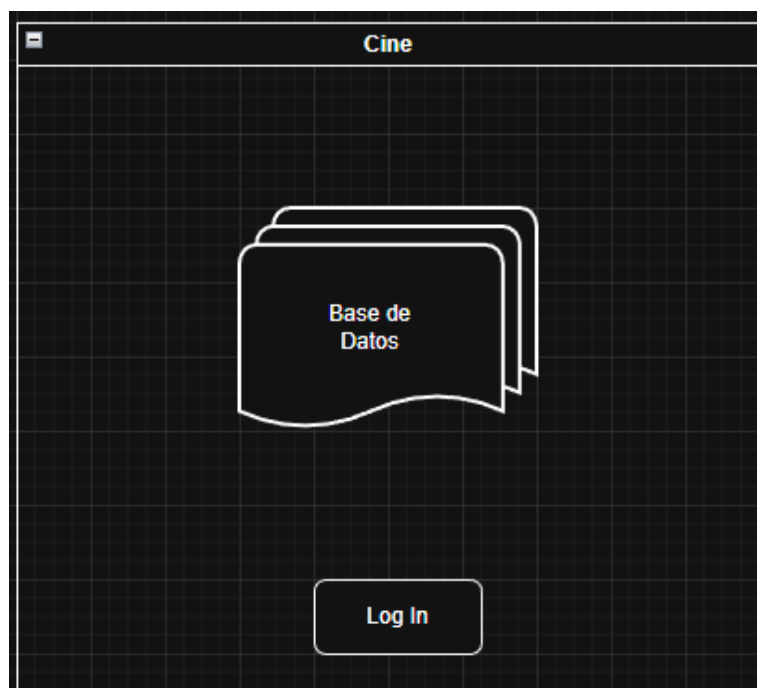
USUARIO BASE DE DATOS

Por motivos de seguridad hemos creado un usuario con los permisos justos y necesarios, y con acceso único a esta base de datos.

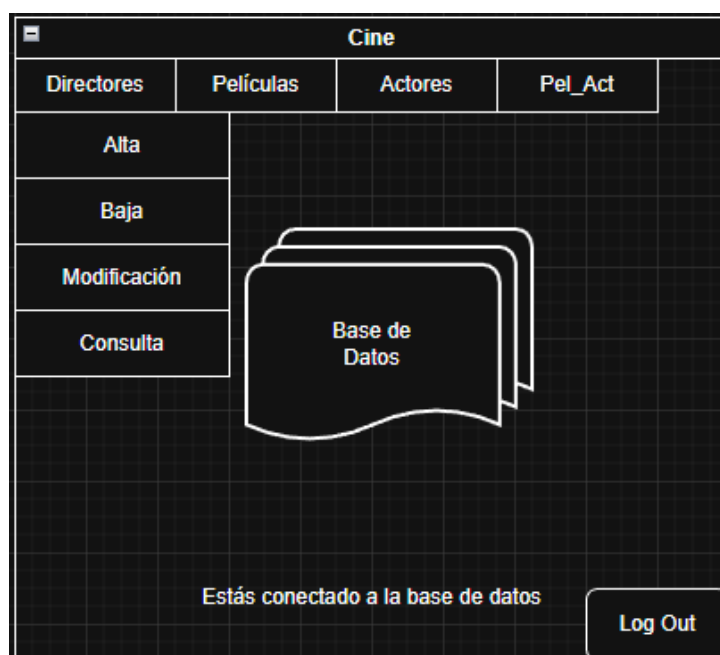
```
53      -- Crear usuario Select, Insert, Delete y Update (CRUD) localhost
54 • CREATE USER 'studium'@'localhost' IDENTIFIED BY 'Studium2025#';
55 • GRANT SELECT, INSERT, DELETE, UPDATE ON p_cine.* TO 'studium'@'localhost';
56 • FLUSH PRIVILEGES;
```

DISEÑO FUNCIONAL

Principal



Principal2



Login



The Login form is displayed within a window titled "Login". It features a dark background with a light grid. The instructions "Introduzca sus datos de usuario para iniciar sesión" are centered at the top. Below this, there are two input fields: one for "Usuario" and one for "Clave". The "Clave" field is masked with asterisks. At the bottom, there are two buttons: "Iniciar" on the left and "Limpiar" on the right.

Login	
Introduzca sus datos de usuario para iniciar sesión	
Usuario	Usuario
Clave	*****
Iniciar	Limpiar

Directores – Alta



The "Directores - Alta" form is shown in a window with the same title. It has a dark background with a light grid. At the top, there is a navigation bar with four tabs: "Directores", "Películas", "Actores", and "Pel_Act". The "Directores" tab is selected. The main heading "Introduzca un nuevo director" is centered. Below it, there are three input fields for "Nombre", "Apellidos", and "Nacionalidad". At the bottom, there are two buttons: "Aceptar" on the left and "Limpiar" on the right.

Directores - Alta			
Directores	Películas	Actores	Pel_Act
Introduzca un nuevo director			
Nombre			
Apellidos			
Nacionalidad			
Aceptar	Limpiar		

Directores – Baja

Directores - Bajas

Directores

Películas

Actores

Pel_Act

¿Qué director desea eliminar?

Elige un director...

↓

Eliminar

¿Segur@?

Se va a eliminar a "TEXT"

¿Estás seguro de eliminarlo?

Sí

No

Directores – Modificación

Directores - Modificación

Directores

Películas

Actores

Pel_Act

¿Qué director desea modificar?

Elige un director...

↓

Editar

Directores - Modificación

Directores

Películas

Actores

Pel_Act

Estás editando: Director

Nombre

NombreDirector

Apellidos

ApellidosDirector

Nacionalidad

NacionalidadDirector

Aceptar

Limpiar

Directores – Consulta

Directores - Consulta

Directores

Películas

Actores

Pel_Act

Text	Text	Text	Text	Text

Consultar

Exportar a PDF

Películas – Alta

Películas - Alta

Directores

Películas

Actores

Pel_Act

Introduzca una nueva película

Título

Género

Fecha de Estreno
AA - MM - DD

Director

Seleccionar un director..

↓

Aceptar

Limpiar

Películas – Baja

Películas - Baja

Directores

Películas

Actores

Pel_Act

¿Qué película desea eliminar?

Elige una película...

↓

Eliminar

¿Segur@?

Se va a eliminar a "TEXT"

¿Estás seguro de eliminarlo?

Sí

No

Películas – Consulta

Películas - Consulta

Directores

Películas

Actores

Pel_Act

Text	Text	Text	Text	Text

Consultar

Exportar a PDF

Actores – Alta

Actores - Alta

Directores

Películas

Actores

Pel_Act

Introduzca un nuevo actor

Nombre

Apellidos

Salario
(Números)

Aceptar

Limpiar

Actores – Baja

Actores - Baja

Directores

Películas

Actores

Pel_Act

¿Qué actor desea eliminar?

Elige una actor...

↓

Eliminar

¿Segur@?

Se va a eliminar a "TEXT"

¿Estás seguro de eliminarlo?

Sí

No

Actores – Modificación

Actores - Modificación

Directores

Películas

Actores

Pel_Act

¿Qué actor desea modificar?

Elige un actor...

↓

Editar

Actores - Modificación

Directores

Películas

Actores

Pel_Act

Estás editando: Actor

Nombre

NombreActor

Apellidos

ApellidosActor

Salario (Números)

00.00

Aceptar

Limpiar

Actores - Consulta

Text	Text	Text	Text	Text

Consultar

Exportar a PDF

Diálogos

Error

Error de TEXT

Enhorabuena

Se ha realizado el TEXT con éxito

Acceso denegado

Esta parte está en desarrollo

/ Todos estos son bocetos de diseño que no reflejan la versión final en la que se han ido aplicando mejoras y corrigiendo errores /

REPOSITORIO GITHUB

En [este](#) enlace se encuentra el repositorio dónde he llevado a cabo el proyecto, con todos los archivos que he ido usando para el desarrollo de la aplicación.

EJECUCIÓN Y PRUEBAS

Cuando ejecutemos nuestra aplicación comenzaremos desde la clase JAVA Principal, que nos mostrará un Frame con un botón de LogIn, desde ahí al presionarlo nos enviará a la clase Login, que ejecuta otro Frame en el que debemos escribir los datos de inicio de sesión, esto realizará un SELECT a nuestra Base de Datos pasando varias comprobaciones para ver si coincide con alguno.

Actualmente disponemos de dos usuarios: “Usuario” que es de tipo “basico” y solo dispone de permisos para realizar altas (INSERT INTO) y de “Administrador” que es de tipo admin y dispone de todos los permisos del CRUD (sentencias SQL INSERT INTO, SELECT, UPDATE, DELETE). La clave está encriptada por motivos de seguridad.

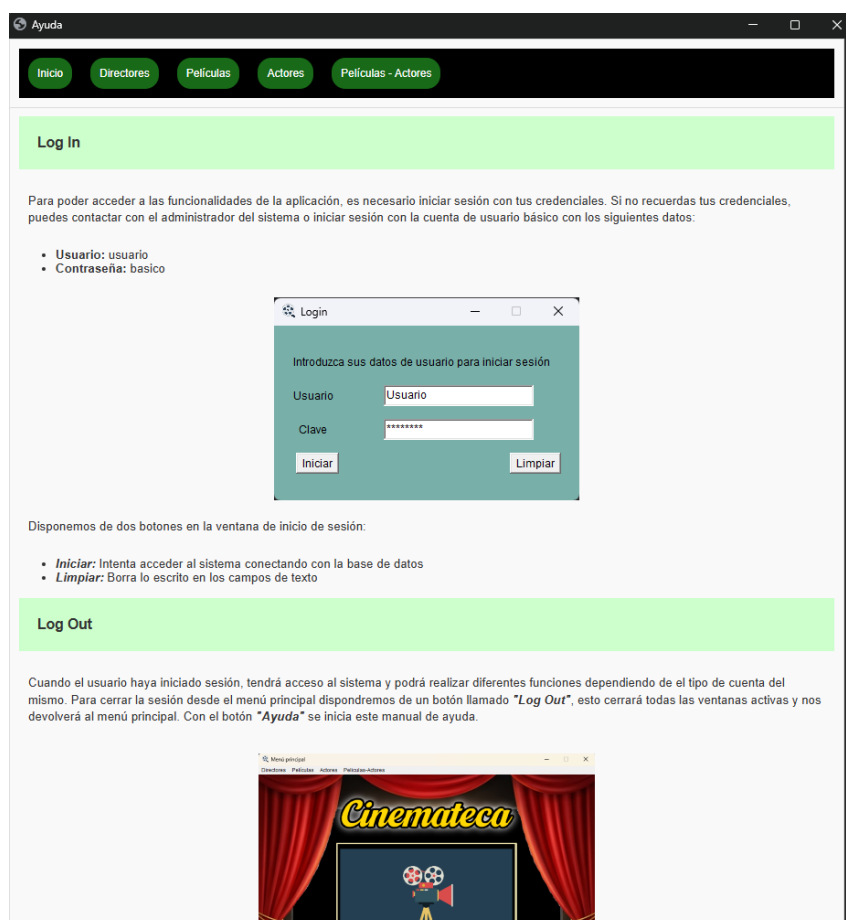


	idUsuario	nombreUsuario	claveUsuario	tipoUsuario
▶	1	Administrador	ab20f6e3ee225e70bdeb0363708de19b44a7e1...	admin
	2	Usuario	b1bbef3b6a1cb6f98a451620e6b59f6329e17fa...	basico

Si escribimos los datos incorrectos nos saltará un Dialog indicando que ese usuario no existe en nuestra base de datos.

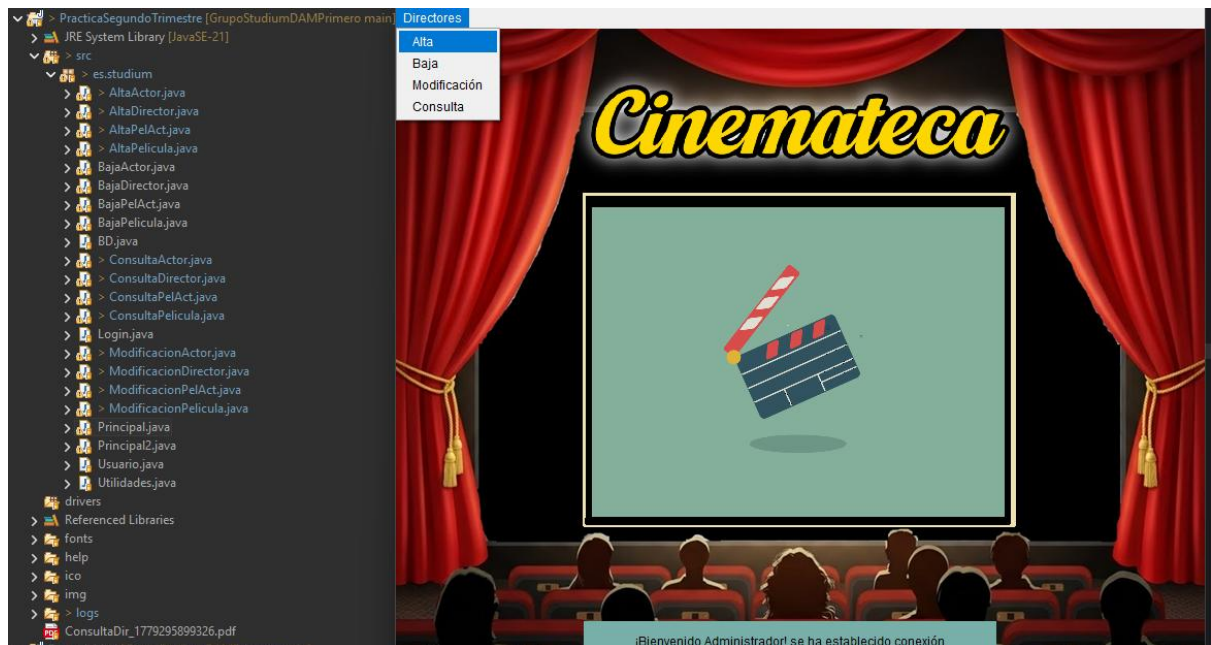


Si el usuario se equivoca tres veces al escribir las credenciales al siguiente intento se le abrirá el manual de Ayuda, en el que aparecen los datos para acceder al sistema con una cuenta de usuario básico, aparte de toda información necesaria para el manejo de la aplicación. Este manual también se puede ejecutar presionando el botón del menú principal tras iniciar en el sistema.



De otro modo si escribimos las credenciales de uno de los usuarios de nuestra Base de Datos accederemos a la parte funcional de nuestro programa.

Acceso Administrador



Acceso Usuario



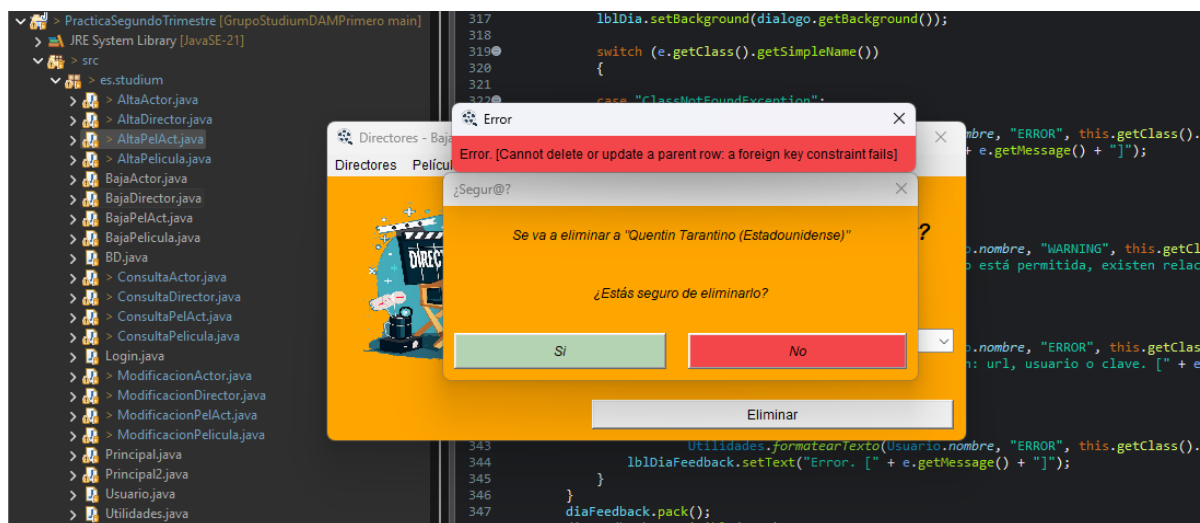
En la barra de menú superior tenemos varias opciones, cuando presionamos una de ellas nos ejecutará la clase JAVA correspondiente, lo que abrirá un nuevo Frame.

Tenemos varias verificaciones en nuestro programa, si realizamos cualquiera de las operaciones del CRUD de forma incorrecta, nos saltará un Dialog con el mensaje de error, por otro lado, si todo sale bien tendremos una confirmación de que la operación se ha llevado a cabo con éxito.

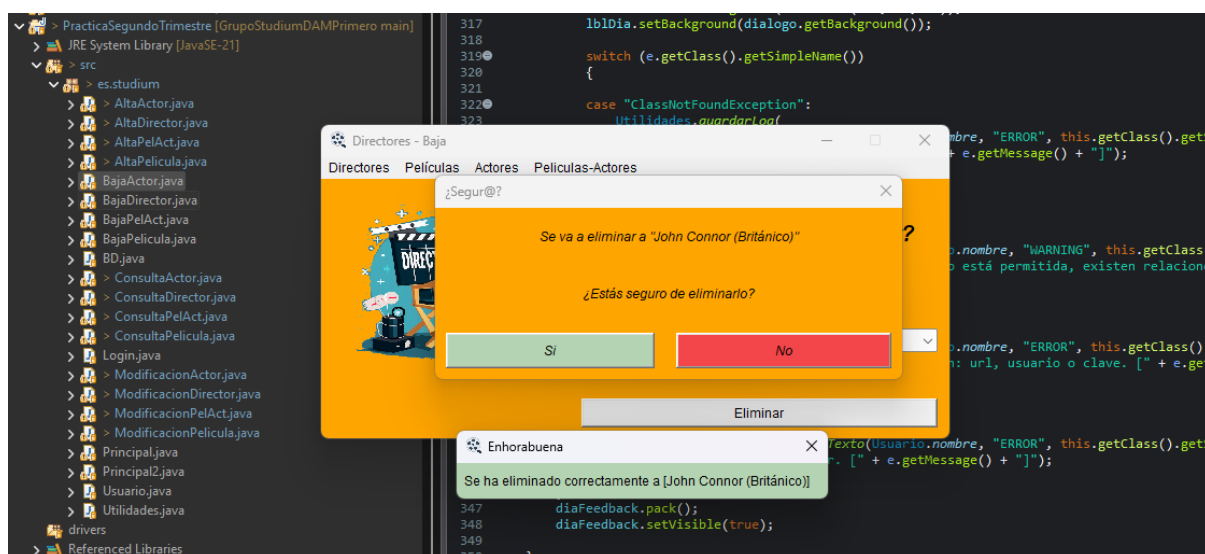


En el caso de las bajas hay que decir que tenemos un Dialog de confirmación, y cabe destacar que para las tablas que están relacionadas, los registros que contengan un "Foreign Key" de otra tabla si los intentamos eliminar saltará un aviso de error, ya que por nuestra configuración de la Base de Datos no es posible, si quisiésemos habilitar esto deberíamos cambiar la configuración de "onDelete" y "onUpdate" a "Cascade".

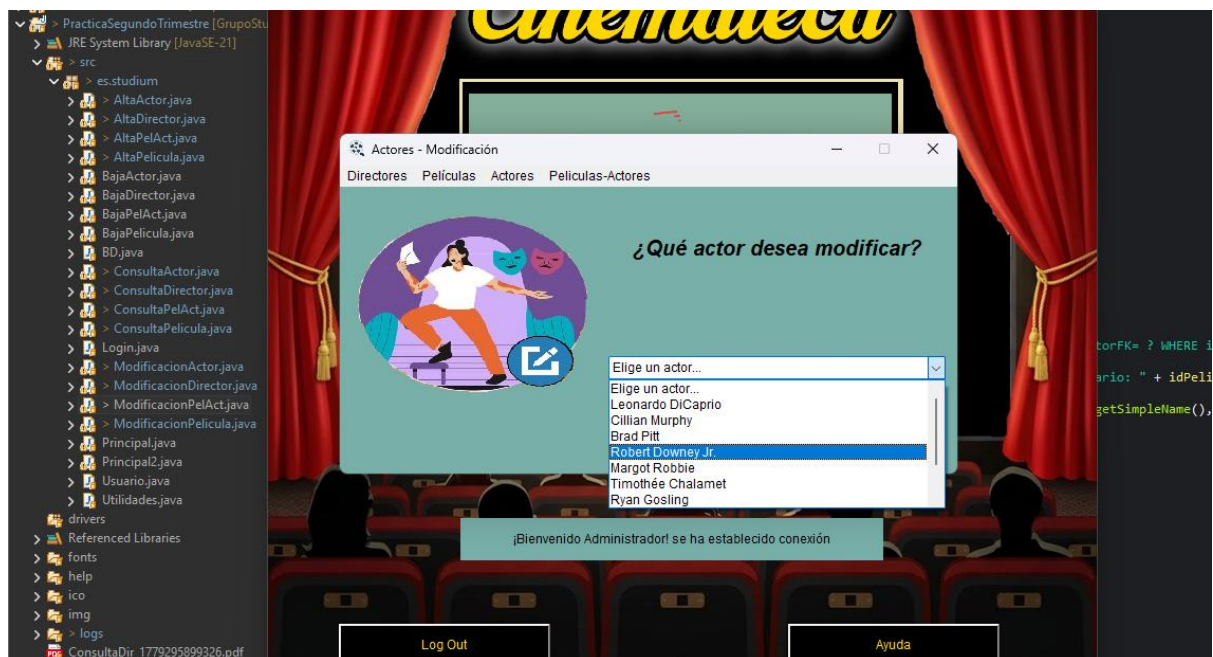
Para cada error hay un método creado que captura las excepciones y nos muestra el error en concreto.



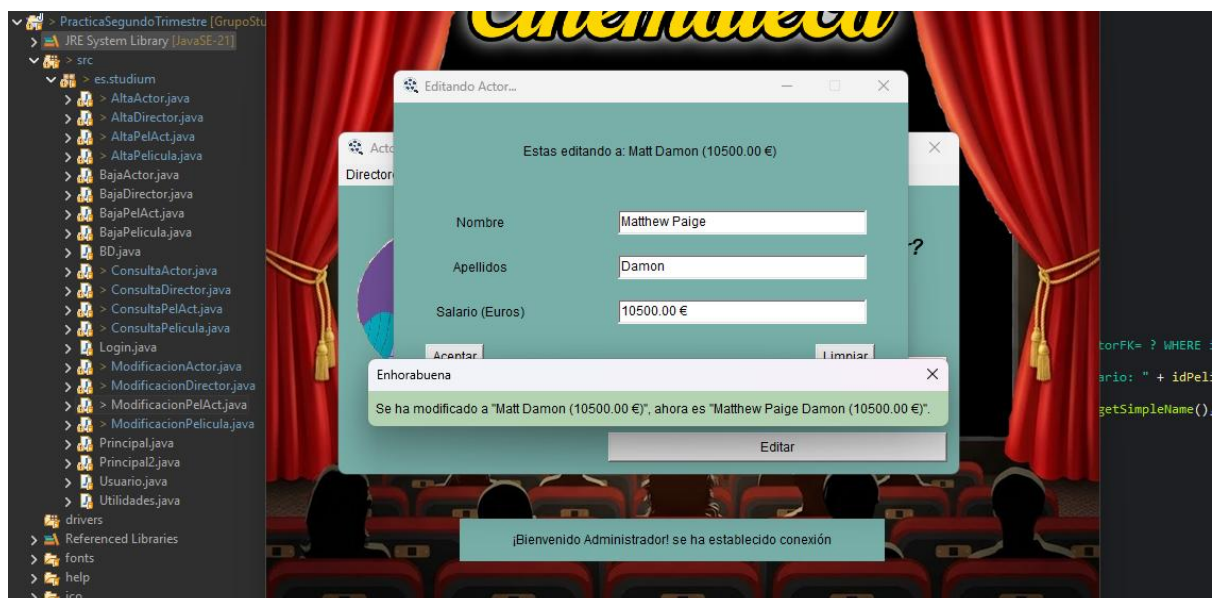
En cambio, si realizamos una baja con éxito tendremos nuestro mensaje de confirmación.



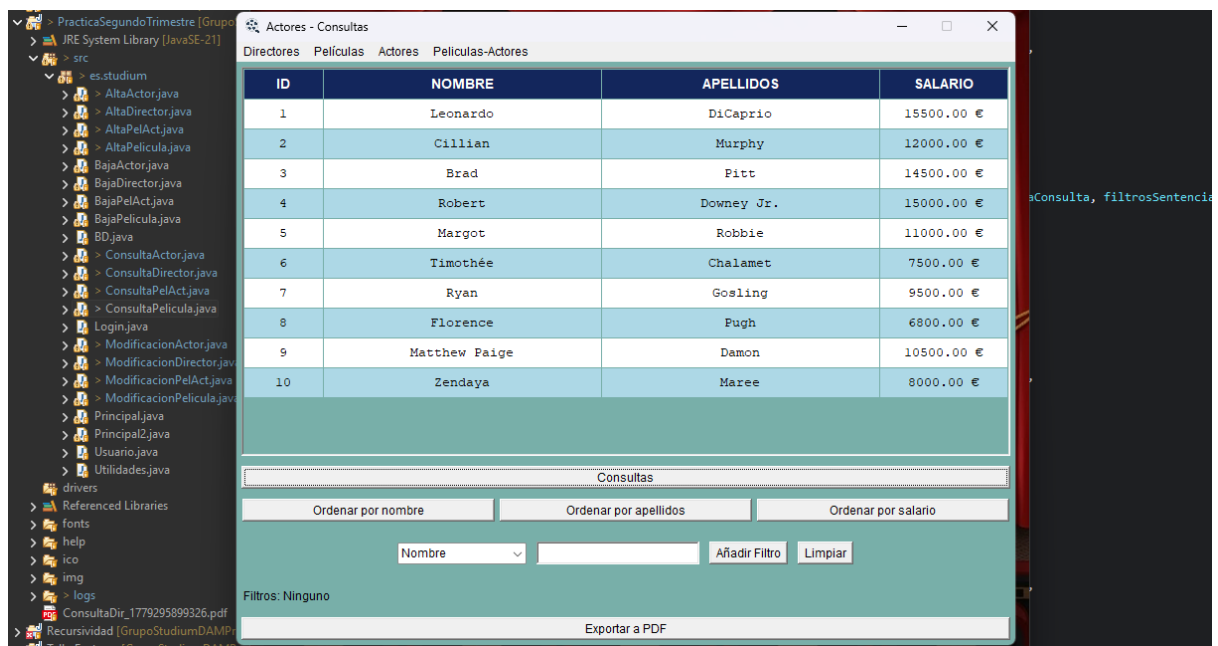
Para las modificaciones, igual que para las bajas disponemos de un Choice que nos permite elegir el elemento que queramos modificar.



La modificación se actualizará a tiempo real cuando realicemos el UPDATE, mostramos la ejecución con éxito.



Mostramos también la ejecución de una consulta, en la que hemos creado una especie de tabla para que sea más agradable visualmente formada por Labels dentro de un ScrollPane, en la que cada Label actúa como “celda” de la tabla y se añade a la tabla al sacar la información de un SELECT con el Result Set.



La consulta tiene muchas funcionalidades útiles: ordenación, filtros y exportación. La ordenación y el filtrado se basan en modificar la última sentencia de búsqueda realizada añadiendo las cláusulas WHERE y/u ORDER BY.

El método de filtrado está formado por un SWITCH – CASE para que se aplique al campo seleccionado.

```
public void addFiltro() {
    String seleccion = choFiltro.getSelectedIndex();
    String filtro = txtFiltro.getText().trim();

    if (filtro.isEmpty()) {
        return;
    }

    try {
        switch (seleccion) {
            case "Actores":
                sentenciaFiltros.add("(nombreActor LIKE ? OR apellidosActor LIKE ?)");
                filtrosSentencia.add("%" + filtro + "%");
                filtrosSentencia.add("%" + filtro + "%");
                break;

            case "Películas":
                sentenciaFiltros.add("(tituloPelícula LIKE ?)");
                filtrosSentencia.add("%" + filtro + "%");
                break;

            case "Género":
                sentenciaFiltros.add("(generoPelícula LIKE ?)");
                filtrosSentencia.add("%" + filtro + "%");
                break;

            case "Año posterior a":
                Integer.parseInt(filtro);
                sentenciaFiltros.add("(YEAR(fechaEstrenoPelícula) >= ?)");
                filtrosSentencia.add(filtro);
                break;

            case "Año anterior a":
                Integer.parseInt(filtro);
                sentenciaFiltros.add("(YEAR(fechaEstrenoPelícula) <= ?)");
                filtrosSentencia.add(filtro);
                break;
        }

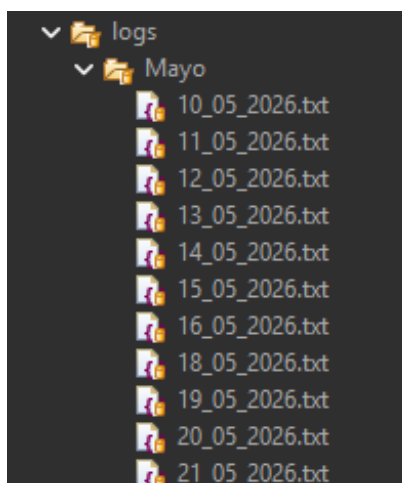
        if (lblFiltrosActivos.getText().equals("Filtros: Ninguno")) {
            lblFiltrosActivos.setText("Filtros: " + seleccion + "(" + filtro + ")");
        } else {
            lblFiltrosActivos.setText(lblFiltrosActivos.getText() + ", " + seleccion + "(" + filtro + ")");
        }
        txtFiltro.setText("");
    } catch (NumberFormatException nfe) {
        dialogoComprobacion(new Exception("Para el año debe introducir un número válido (ej: 1995)."));
    }
}
```


Para la exportación, hemos implementado una librería externa llamada iText añadiendo los archivos .jar necesarios al ClassPath de nuestro proyecto. Esta librería sirve para construir el PDF bloque a bloque usando objetos Pdf, Document, Table, Cell entre los más destacados.

Listado de Actores

ID	Nombre	Apellidos	Salario
1	Leonardo	DiCaprio	15500.00 €
2	Cillian	Murphy	12000.00 €
3	Brad	Pitt	14500.00 €
4	Robert	Downey Jr.	15000.00 €
5	Margot	Robbie	11000.00 €
6	Timothée	Chalamet	7500.00 €
7	Ryan	Gosling	9500.00 €
8	Florence	Pugh	6800.00 €
9	Matthew Paige	Damon	10500.00 €
10	Zendaya	Maree	8000.00 €

Se ha añadido una carpeta de “Logs” organizada por día y mes en la que se guarda la información de todos los movimientos de cada usuario que acceda al sistema. La estructura del archivo de log intenta recopilar la máxima información posible, este recoge el momento en el que se realizó la acción, el usuario que la realizó, la ventana en la que se encontraba y la propia acción que ejecutó. Además, si el usuario tuvo que seleccionar o escribir algún dato también se recoge en los archivos de logs.



```

1 [22/05/2026 08:53:24] [Administrador] [INFO] [Login] Inicio de sesión
2 [22/05/2026 08:53:57] [Administrador] [INFO] [AltaPelícula] Formato de fecha incorrecto
3 [22/05/2026 08:53:57] [Administrador] [ERROR] [AltaPelícula] Formato de fecha incorrecto
4 [22/05/2026 08:54:16] [Administrador] [INFO] [AltaPelícula] Sentencia: INSERT INTO peliculas (tituloPelícula, generoPelícula, fechaEstrenoPelícula, idDirectorFK) VALUES (?, ?,
5 Valores seleccionados por el usuario: Mortadelo y Filemón, Comedia, 09-04-2007, Alba Duino (Italiana)
6 [22/05/2026 08:54:26] [Administrador] [INFO] [BajaActor] DELETE from actores WHERE idActor = ?
7 Valor seleccionado por el usuario: 15 = Ben White
8 [22/05/2026 08:54:40] [Administrador] [INFO] [ConsultaDirector] SELECT * FROM directores | Filtros: []
9 [22/05/2026 08:54:48] [Administrador] [INFO] [Principal2] Apertura de manual de ayuda
10 [22/05/2026 08:54:55] [Administrador] [INFO] [ConsultaDirector] PDF Generado con la última consulta realizada
11 [22/05/2026 08:55:00] [Administrador] [INFO] [Principal2] Sesión cerrada
12

```

Las conexiones a la base de datos la controlamos desde la clase .java BD de nuestra aplicación, en ella tenemos también organizadas las sentencias que por motivos de seguridad usamos con Prepared Statement.

```
public class BD {

    public static String driver = "com.mysql.cj.jdbc.Driver";
    public static String url = "jdbc:mysql://localhost:3306/p_cine";
    public static String usuario = "studium";
    public static String clave = "Studium2025#";

    public static String consultaSQLUsuarios = "SELECT * FROM usuarios";

    public static String eliminarSQLDirector = "DELETE from directores WHERE idDirector = ?";
    public static String eliminarSQLPelicula = "DELETE from peliculas WHERE idPelicula = ?";
    public static String eliminarSQLActor = "DELETE from actores WHERE idActor = ?";
    public static String eliminarSQLPelAct = "DELETE from peliculas_actores WHERE idPeliculaActor = ?";

    public static String consultaSQLDirectores = "SELECT * FROM directores";

    public static String consultaSQLActores = "SELECT idActor, nombreActor, apellidosActor, CONCAT(salari";

    public static String consultaSQLPel = "SELECT * FROM peliculas";
    public static String consultaSQLPeliculas = "SELECT idPelicula, tituloPelicula, generoPelicula, DATE_";
    public static String consultaSQLPeliculas2 = "SELECT idPelicula, tituloPelicula, generoPelicula, YEAR";
    public static String consultaSQLPeliculas3 = "SELECT idPelicula, tituloPelicula, generoPelicula, DATE";

    public static String consultaSQLPelAct = "SELECT idPeliculaActor, CONCAT(tituloPelicula, ' (', YEAR(f";
    public static String consultaSQLPelAct2 = "SELECT idPeliculaActor, idPelicula, idActor, CONCAT(titulo";

    public static Connection connection = null;
    public static Statement statement = null;
    public static ResultSet rs = null;
    public static PreparedStatement ps = null;

    public static void conectarBD() throws ClassNotFoundException, SQLException {
        Class.forName(driver);
        connection = DriverManager.getConnection(url, usuario, clave);
        statement = connection.createStatement();
    }

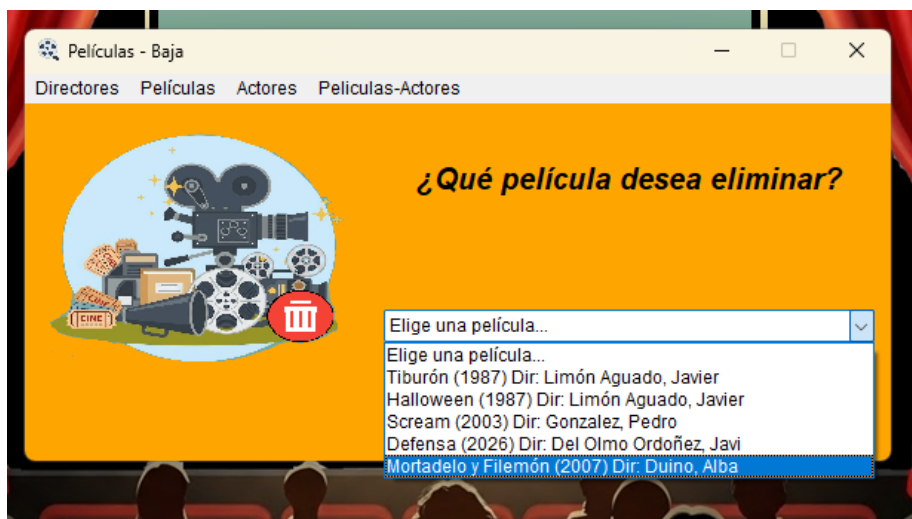
    public static void desconectarBD() throws SQLException {
        if (connection != null) {
            connection.close();
        }
    }
}
```

Como estamos trabajando con base de datos MySQL, se basa en las relaciones entre tablas, y gracias al uso de las “Foreign Keys” para evitar la redundancia de datos, podemos enlazar diferentes tablas usando sentencias con JOIN, esto garantiza la integridad y consistencia de los datos.

```
AS 'nombreCompletoDirector' FROM peliculas JOIN directores ON directores.idDirector = peliculas.idDirectorFK";
reCompletoDirector' FROM peliculas JOIN directores ON directores.idDirector = peliculas.idDirectorFK";
) AS 'nombreCompletoDirector', idDirector FROM peliculas JOIN directores ON directores.idDirector = peliculas.idDirectorFK";

peliculas_actores ON peliculas_actores.idPeliculaFK = peliculas.idPelicula JOIN actores ON actores.idActor = peliculas_actores.idA
FROM peliculas JOIN peliculas_actores ON peliculas_actores.idPeliculaFK = peliculas.idPelicula JOIN actores ON actores.idActor =
```

Podemos observar que haciendo un JOIN de la tabla Directores con la tabla Películas podemos sacar el nombre y los apellidos del director relacionado por idDirectorFK perteneciente a la Película.



REFERENCIAS

- Apuntes de clase
- [GitHub](#)
- [Documentación Oracle](#)